DTI201 • ต่อจากสไลด์ Express API

จาก Express API สู่ MongoDB

เพิ่มหนังสือแล้วปิดเซิร์ฟเวอร์: ข้อมูลจะยังอยู่ไหม?





MERN • ใช้โปรเจกต์จาก Lab 09 ต่อ

หนังสือบทที่ 9: แนวคิดฐานข้อมูล | Lab 10: MongoDB + Mongoose

แผนที่คาบเรียน

จากหนังสือที่หาย → หนังสือที่อยู่ต่อ

เริ่มจาก API ที่เราทำแล้วใน Lab 09 แล้วเพิ่มที่เก็บข้อมูลให้แอป MERN

01
ดูปัญหา
Restart Express แล้วหนังสือหาย

02
เข้าใจ MongoDB
Collection และ document

03
สร้าง Model
Mongoose schema และ seed

04
ต่อ API
GET และ POST

05
พิสูจน์ผล
Restart และดู React

เป้าหมายที่ต้องสาธิตได้

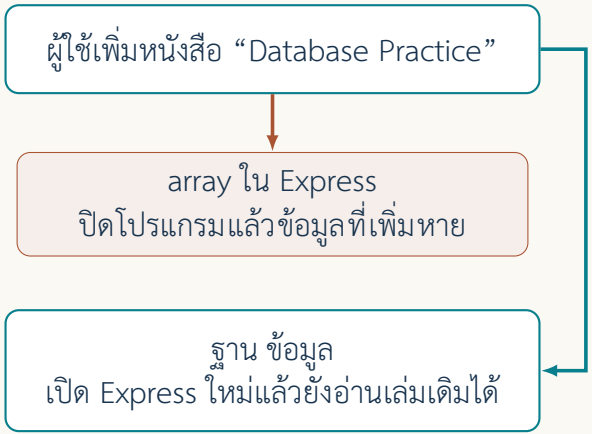
POST หนังสือ → ปิด และ เปิด Express ใหม่ → GET ยัง พบ หนังสือ เล่ม เดิม

ฐานข้อมูลคืออะไร?

ฐานข้อมูล (database) คือระบบที่เก็บข้อมูลเพื่อให้โปรแกรมเพิ่ม ค้นหา แก้ไข และนำกลับมาใช้ได้

ใน Lab 09 เราเก็บหนังสือไว้ใน array ของ Express. เมื่อหยุดโปรแกรม หนังสือที่เพิ่มระหว่างรันหายไป

คาบนี้เราย้ายหนังสือไปเก็บในฐานข้อมูล. ปิด Express แล้วเปิดใหม่ แอปจึงอ่านหนังสือเล่มเดิมกลับมาได้



ลองคิดหรือทดลอง: หลังปิด Express หนังสือที่เพิ่มเองอยู่ต่อได้ ต้องเก็บไว้ที่ไหน?

ฐานข้อมูลมีหลายแบบ

ฐานข้อมูลแต่ละแบบจัดข้อมูลต่างกัน. วันนี้ให้รู้จักสี่แบบจากคำถามว่า “ข้อมูลหนึ่งชิ้นหน้าตาอย่างไร?”

เชิงสัมพันธ์ (relational): ตารางและแถว เช่นตัวอย่าง PostgreSQL ในหนังสือ

เอกสาร (document): เอกสารที่มีชื่อฟิลด์และค่า เช่น MongoDB

คีย์–ค่า (key–value): ใช้คีย์หาค่า เช่นข้อมูลชั่วคราวของ session

กราฟ (graph): เน้นความสัมพันธ์ระหว่างสิ่งต่าง ๆ

ตาราง
แถวหนังสือ

เอกสาร
Book หนึ่งเล่ม

คีย์–ค่า
session → ผู้ใช้

กราฟ
คน ↔ หนังสือ

Lab นี้เลือกแบบเอกสาร: MongoDB

ลองคิดหรือทดลอง: หนังสือหนึ่งเล่มใน Lab 10 จะเก็บเป็นแถวของตารางหรือ document?

ข้อมูลหนังสือเดียวกัน จัดได้สองรูปแบบ

สมมติเรามีหนังสือ “Web Basics” โดย Nok. สิ่งที่ต้องจำคือชื่อ ผู้แต่ง และชนิดแหล่งเรียนรู้

ฐานข้อมูลเชิงสัมพันธ์จัดเป็น **หนึ่งแถวในตาราง**. ฐานข้อมูลแบบเอกสารจัดเป็น **หนึ่ง document ใน collection**

ทั้งสองแบบเก็บข้อมูลได้. หนังสือใช้แบบแรกเพื่อสอนหลักการ; โปรเจกต์ MERN นี้ใช้แบบหลัง

ตาราง books

id=1	Web Basics	Nok	book

collection books

```
{title:"Web Basics",
author:"Nok", sourceType:"book"}
```

ลองคิดหรือทดลอง: คำว่า “แถว” ในหนังสือ เทียบกับคำใดใน Lab นี้?

MongoDB เกิดขึ้นมาอย่างไร?

ทีม 10gen เริ่มพัฒนา MongoDB ในปี **2007** เพื่อทดลองวิธีเก็บข้อมูลที่ยืดหยุ่นสำหรับเว็บแอปขนาดใหญ่

MongoDB รุ่นแรกเผยแพร่ในปี **2009**. ชื่อมาจากคำว่า “humongous” ที่สื่อถึงข้อมูลจำนวนมาก

ในปี **2013** บริษัทเปลี่ยนชื่อจาก 10gen เป็น MongoDB. วันนี้เราจะใช้ MongoDB เป็นฐานข้อมูลของโปรเจกต์ MERN

อ่านที่มา: MongoDB Our Story



แนวคิดหลัก: เก็บข้อมูลเป็น document ที่แอปอ่านและเขียนได้

ลองคิดหรือทดลอง: เหตุใดประวัตินี้จึงทำให้เราเรียก MongoDB ว่าฐานข้อมูลแบบ document?

MongoDB มีลักษณะสำคัญอะไร?

Document เก็บฟิลด์และค่าไว้ด้วยกัน คล้าย object ที่เราใช้ใน JavaScript. MongoDB เก็บจริงในรูป BSON

Collection รวม documents ชนิดเดียวกัน. ฟิลด์ของเอกสารปรับได้ แต่แอปของเรายังต้องตรวจว่าหนังสือมีข้อมูลที่จำเป็น

Index ช่วยค้นข้อมูลตามฟิลด์ และ unique index ช่วยห้ามชื่อหนังสือซ้ำ. ข้อมูลอยู่ในฐานข้อมูล แม้เรา restart Express

Document + BSON
ข้อมูล หนังสือ หนึ่ง เล่ม

Collection
รวม หนังสือ หลาย เล่ม

Index
ค้นหา และ ตรวจ ชื่อ ซ้ำ

แอปยังต้องตรวจข้อมูลหนังสือก่อนบันทึก

ลองคิดหรือทดลอง: “ฟิลด์ปรับได้” หมายความว่าเราไม่ต้องตรวจข้อมูลอีกแล้วหรือไม่?

MERN แต่ละตัวทำงานตรงไหน?

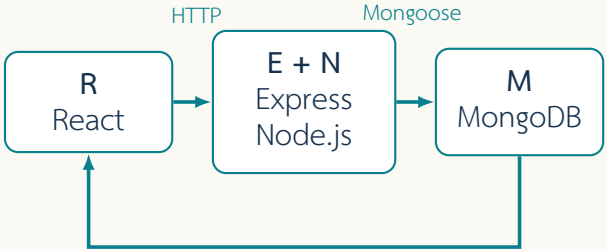
M = MongoDB เก็บหนังสือที่ต้องอยู่ต่อ

E = Express รับคำขอ GET/POST ตรวจข้อมูล และตอบ HTTP

R = React แสดงหน้าเว็บและรับการกดปุ่ม

N = Node.js รันโค้ด Express ฝั่งเซิร์ฟเวอร์

Mongoose ไม่ใช่ตัวอักษรใน MERN; เป็นเครื่องมือให้โค้ด Node.js ติดต่อ MongoDB และกำหนด Book model



เราเขียนทั้งสี่ส่วนให้ทำงานร่วมกันใน Course Book Explorer

ลองคิดหรือทดลอง: เมื่อผู้ใช้กด “เพิ่มหนังสือ” ส่วนใดรับการกด และส่วนใดเก็บข้อมูล?

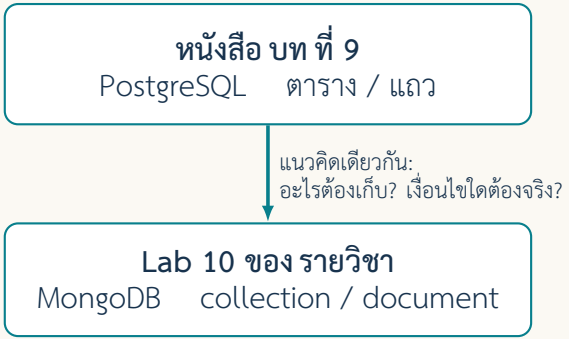
หนังสือใช้ PostgreSQL แต่คาบนี้ใช้ MongoDB

เหตุใดชื่อฐานข้อมูลจึงต่างกัน?

หนังสือบทที่ 9 ใช้ PostgreSQL และตัวอย่างร้านกาแฟเพื่อสอนวิธีคิดเรื่องข้อมูลที่ต้องเก็บ เงื่อนไขของข้อมูล และการค้นหา

รายวิชานี้ทำเว็บแอปแบบ **MERN**: MongoDB, Express, React และ Node.js. เราจึงใช้ MongoDB เก็บหนังสือใน Lab 10

อ่านหนังสือเพื่อเข้าใจ **คำถามที่ต้องตอบ** แล้วทำ Lab ด้วย **เครื่องมือของโปรเจกต์นี้**. ไม่ต้องพิมพ์ SQL ในคาบนี้



MERN ระบุฐานข้อมูลที่ใช้จริง: MongoDB

ลองคิดหรือทดลอง: ถ้าเห็นคำว่า “ตาราง” ในหนังสือ วันนี้เราควรมองหาอะไรใน MongoDB?

ช่วงที่ 1 / 4

ข้อมูลหายไปไหน?

ลองจากโปรเจกต์ Express ที่นักศึกษาทำเองเมื่อคาบก่อน

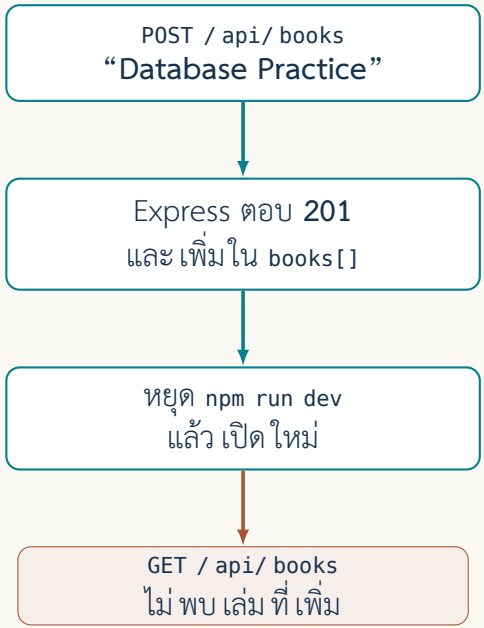
React → Express → MongoDB → Express → React

201 Created แล้วข้อมูลอยู่ที่ไหน?

ใน Lab 09 route `POST /api/books` ตรวจข้อมูล เพิ่มหนังสือลง array แล้วตอบ `201`

แต่ array อยู่ในหน่วยความจำของกระบวนการ Node.js. เมื่อหยุด Express กระบวนการจบและ array ที่เพิ่มระหว่างรันก็หาย

ก่อนลอง ให้เขียนว่า `GET /api/books` หลัง restart จะพบเล่มที่เพิ่งเพิ่มหรือไม่



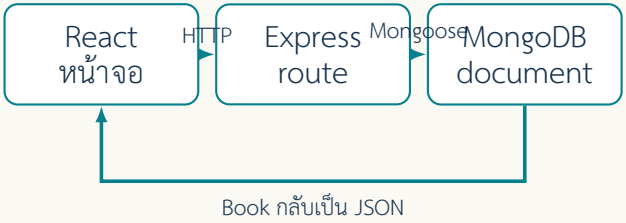
ลองคิดหรือทดลอง: `201` บอกว่า route ทำงานสำเร็จ หรือบอกด้วยว่าข้อมูลจะอยู่หลังปิดโปรแกรม?

เปลี่ยนที่เก็บ โดยตามเส้นทางเดิม

หน้า React ยังส่ง `GET` และ `POST` ไปที่ `/api/books` Express ยังตรวจข้อมูลและส่ง status กับ JSON กลับ

สิ่งที่เพิ่มคือ MongoDB หลัง Express. Mongoose ช่วยกำหนดรูปของ Book และส่งคำสั่งอ่านหรือเขียน document

เปิด `app.js` ของตนเองแล้วชี้จุดที่ route ใช้ array; จุดนั้นจะเปลี่ยนเป็นการเรียก `book-store.js`



`GET /api/books` และ `POST /api/books`
ยังเป็นเส้นทางที่หน้า React เรียก

ลองคิดหรือทดลอง: จุดใดในแผนภาพเป็นที่เก็บข้อมูลถาวร? จุดใดเป็นผู้ส่งคำตอบ HTTP?

ช่วงที่ 2 / 4

หนังสือหนึ่งเล่มใน MongoDB

มอง document จริงก่อนเขียน Mongoose schema

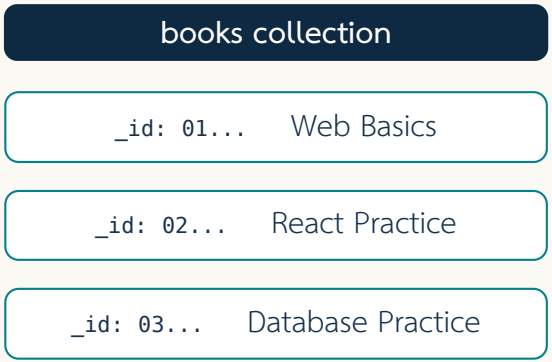
React → Express → MongoDB → Express → React

Collection และ document คืออะไร?

Collection คือกลุ่มข้อมูลชนิดเดียวกัน เช่น `books`

Document คือข้อมูลหนึ่งชิ้นในกลุ่มนั้น เช่นหนังสือ “Web Basics” พร้อมชื่อผู้แต่งและ summary

หนังสือใช้ตารางกับแถวเพื่ออธิบายโจทย์เดียวกัน: เราต้องรู้ว่าแต่ละข้อมูลหมายถึงอะไรและจะหามันกลับมาอย่างไร



หนังสือ: ตาราง / แถว Lab: collection / document

ลองคิดหรือทดลอง: ถ้าต้องเก็บหนังสืออีกหนึ่งเล่ม เราเพิ่ม collection ใหม่หรือ document ใหม่?

ดู document หนึ่งเล่มก่อนสร้าง model

ข้อมูลที่เราต้องใช้แสดงบนหน้า React คือ `title`, `author`, `sourceType` และ `summary`

MongoDB สร้าง `_id` แอปจะมี `titleKey` สำหรับกันชื่อซ้ำโดยไม่สนตัวพิมพ์ใหญ่เล็ก และมีเวลา `createdAt/updatedAt`

`titleKey` เป็นข้อมูลภายใน ไม่ให้ผู้ใช้กรอกและไม่ส่งไปหน้า React

```
{
_id: ObjectId("..."),
title: "Web Basics",
author: "Nok",
sourceType: "book",
summary: "Web foundation",
titleKey: "web basics"
}
```

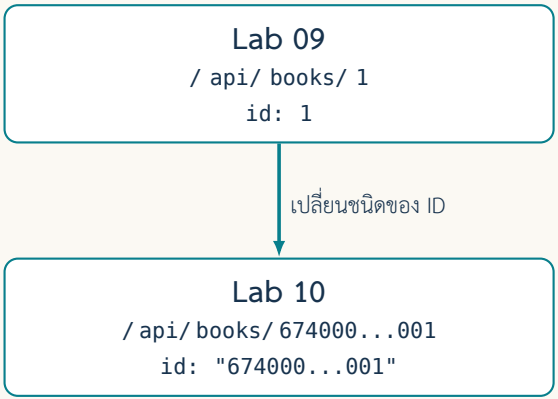
ลองคิดหรือทดลอง: ฟิลด์ใดเป็นข้อมูลภายในฐานข้อมูล และฟิลด์ใดควรแสดงต่อผู้ใช้?

เลข ID เดิม เปลี่ยนเป็น ObjectId

Lab 09 ใช้เลข `1, 2` เพราะเราสร้าง array เอง. MongoDB สร้าง **ObjectId** ให้ document

API ยังใช้เส้นทาง `/api/books/:id` แต่ค่า ID เปลี่ยนเป็นข้อความ 24 อักขระ เช่น `674000000000000000000001`

ใน API แปลง `_id` เป็น `id: String(doc._id)`. React ที่ใช้ `id: string` อยู่แล้วใช้ต่อได้ แต่การตรวจ “จำนวนเต็มบวก” จาก Lab 09 ต้องเปลี่ยน



ลองคิดหรือทดลอง: `/api/books/1` หลังเปลี่ยนเป็น MongoDB ควรตอบ 200, 404 หรือ 422 เพราะอะไร?

Schema ระบุเงื่อนไขของ Book

Mongoose schema บอกว่า Book ต้องมีข้อมูลอะไร: `title`, `author`, `sourceType`, `summary`

กำหนด `title` ยาวอย่างน้อย 3 ตัวอักษร และ `sourceType` เลือกจาก `book`, `documentation`, `course`

API ยังต้องตรวจ JSON ที่รับมาเอง. Schema ช่วยตรวจเงื่อนไขเมื่อโค้ดส่วนอื่นเขียนข้อมูล

`title: "Web Basics"`
ผ่าน: มีข้อความอย่างน้อย 3 ตัว

`sourceType: "book"`
ผ่าน: อยู่ในรายการที่กำหนด

`sourceType: "video"`
ไม่ผ่านเงื่อนไขของคาบนี้

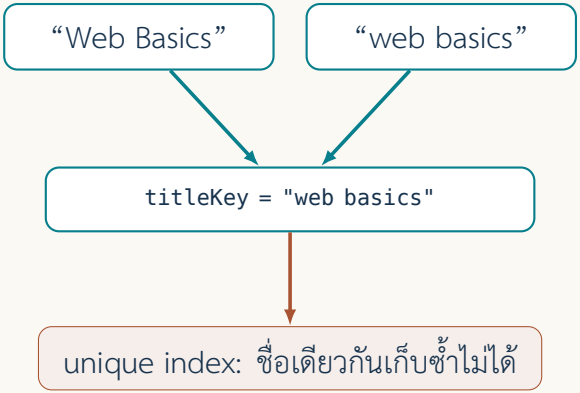
ลองคิดหรือทดลอง: ถ้าชื่อหนังสือเป็นช่องว่างล้วน เราควรตรวจตรงไหนบ้าง?

ชื่อซ้ำต้องให้ MongoDB ช่วยบังคับ

สร้าง `titleKey` จากชื่อที่ตัดช่องว่างและเปลี่ยนเป็นตัวพิมพ์เล็ก เช่น “Web Basics” กับ “web basics” ได้ key เดียวกัน

สร้าง **unique index** บน `titleKey`. ถ้ามีคำขอเพิ่มชื่อเดียวกันพร้อมกัน MongoDB จะรับได้เพียงหนึ่งรายการ

`unique` ใน Mongoose ช่วยสร้าง index แต่ **ไม่ใช่ validator**. Duplicate-key error ต้องแปลงเป็น `409`



ลองคิดหรือทดลอง: ถ้า route ตรวจชื่อซ้ำก่อน INSERT แล้ว ยังจำเป็นต้องมี unique index หรือไม่?

ช่วงที่ 3 / 4

ต่อ MongoDB เข้ากับ Express

ทำงานในโฟลเดอร์ backend ที่เราสร้างเองใน Lab 09

React → Express → MongoDB → Express → React

เปิด MongoDB บนเครื่อง แล้วดูว่าเข้าถึงได้

สร้าง `compose.yaml` ใน backend เดิมเพื่อเปิด MongoDB เฉพาะเครื่องที่ port 27018. ใช้ volume เก็บข้อมูลหลัง container หยุด

เพิ่ม `MONGODB_URI` ใน `.env`; อย่า commit `.env`. ถ้าเครื่องไม่มี Docker ให้ใช้ MongoDB ของห้องเรียนที่ผู้สอนเตรียม

ก่อนเขียน API ให้ใช้ `mongosh` ส่งคำสั่ง `ping` และดู collection

```
docker compose up -d db
```

```
docker compose ps
```

```
docker compose exec db mongosh
```

```
db.runCommand({ping:1}) → ok: 1
```

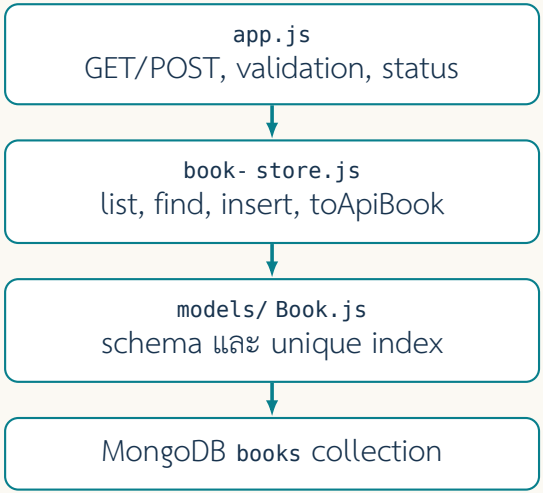
ลองคิดหรือทดลอง: ถ้า `ping` ไม่สำเร็จ ควรแก้การเชื่อมต่อก่อนหรือเริ่มเขียน route ก่อน?

ไฟล์ใดรับผิดชอบงานใด?

`server.js` เชื่อม MongoDB ก่อนเปิด port 3001. `Book.js` ระบุ schema และ index

`book-store.js` อ่าน/เขียน document และแปลง `_id` เป็น `id`. `app.js` ตรวจคำขอและเลือก HTTP status

การแยกนี้ทำให้เราเปลี่ยนที่เก็บข้อมูลโดยยังอ่านเส้นทาง request ใน Express ได้



ลองคิดหรือทดลอง: ถ้าอยากเปลี่ยนข้อความ 409 ที่ client เห็น ควรเริ่มดูไฟล์ใด?

Seed ทำให้ทุกคนเริ่มจากข้อมูลที่ตรวจได้

เขียน `scripts/seed.js` นำหนังสือ 200 เล่มจาก `books.js` ที่ผู้สอนให้มาเข้า MongoDB โดยไม่ใช้เลข ID เดิมเป็น `_id`

ให้รัน seed ซ้ำแล้วไม่เกิดหนังสือซ้ำ โดยใช้ `titleKey` หาเล่มเดิมก่อนเพิ่ม

ตรวจผลใน `mongosh`. หนังสือหนึ่งเล่มในหน้า React ควรตามกลับไปยัง document หนึ่งชิ้นได้

```
npm run seed
```

```
db.books.countDocuments()
```

```
db.books.find().toArray()
```

รัน seed ซ้ำ: จำนวนยังเป็น 200

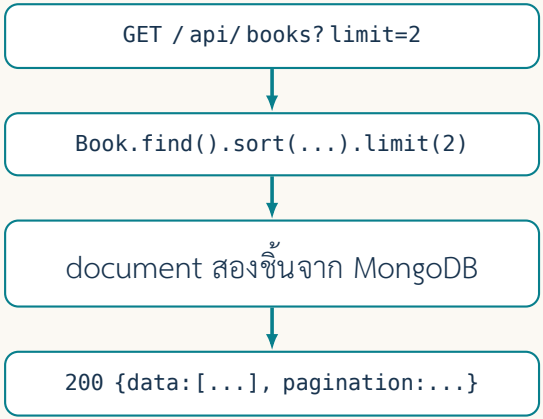
ลองคิดหรือทดลอง: ถ้ารัน seed ครั้งที่สองแล้วจำนวนเพิ่มจาก 200 เป็น 400 เกิดปัญหาที่ใด?

GET รายการ: จาก document กลับเป็น JSON

ใน Lab 09 เราใช้ `books.slice(...)` คาบนี้ store ใช้ `Book.find()` แล้วเรียงด้วย `_id` ก่อนแบ่งหน้า

Express ยังตอบ `{data:[...], pagination: {limit,offset}}` จุดที่ต้องระวังคือ `_id` ของ MongoDB ต้องแปลงเป็น `id` ข้อความ

ใช้ `curl -i` เทียบผลกับ `mongosh` ก่อนเปิด React



ลองคิดหรือทดลอง: ถ้า React แสดง `_id` แทน `id` จุดใดควรแปลง?

GET หนึ่งเล่ม: ID ผิดรูปต่างจากไม่พบ

ก่อนเรียก MongoDB ให้ตรวจว่า `:id` เป็นข้อความ 24 ตัวอักษรฐานสิบหก. ถ้ารูปผิด เช่น `/api/books/1` ให้ตอบ `422`

ถ้ารูปถูกแต่ไม่มี document ให้ตอบ `404`. ถ้าพบให้ตอบ `200` พร้อม `{data:book}`

นี่คือการปรับจาก Lab 09 ที่ตรวจ ID เป็นจำนวนเต็มบวก

`/api/books/1` → **422**
รูป ID ใช้ไม่ได้

ObjectId รูปถูก แต่ไม่มีเล่ม → **404**

ObjectId ของเล่มจริง → **200**

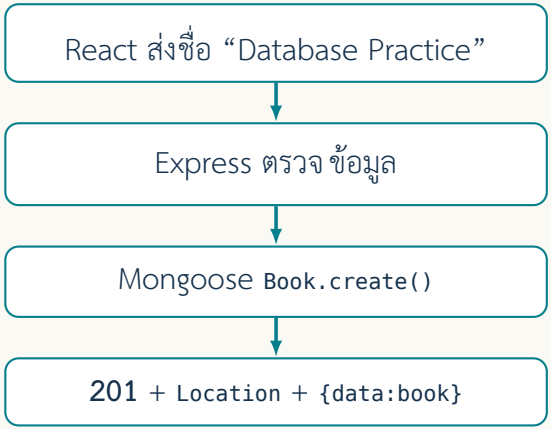
ลองคิดหรือทดลอง: เปิด Lab 09 ของตน: ฟังก์ชันตรวจ ID เดิมอยู่ตรงไหน?

POST หนึ่งเล่ม: สร้าง document แล้วตอบ 201

Express ตรวจ `title`, `author`, `sourceType`, `summary` เหมือนเดิม และไม่รับ `titleKey` จาก client

store คำนวณ `titleKey` แล้วเรียก `Book.create()`. เมื่อสำเร็จ MongoDB คืน document ที่มี `_id`

API แปลงเป็น `id` และตอบ `201` พร้อม `Location: /api/books/<id>`



ลองคิดหรือทดลอง: หลัง POST สำเร็จ จะใช้ค่าใดใน response ไป GET เล่มเดียวกัน?

ข้อผิดพลาดเกิดคนละขั้น

422: JSON อ่านได้ แต่ข้อมูลไม่ผ่านเงื่อนไข เช่นชื่อสั้นเกิน

409: MongoDB unique index ปฏิเสธ `titleKey` ที่ซ้ำ; duplicate-key code คือ `11000`

500: MongoDB ล้ม หรือเกิดข้อผิดพลาดอื่นที่ไม่คาดหมาย. Client ได้ problem JSON กับ request ID ไม่ได้ข้อความเชื่อมต่อหรือรายละเอียดภายใน

ชื่อสั้น → ตรวจข้อมูล → **422**

ชื่อซ้ำ → unique index → **409**

ฐานข้อมูลหยุด → error handler → **500**

ทุกกรณีบันทึก status และ request ID เพื่ออธิบายผล

ลองคิดหรือทดลอง: `unique: true` ใน Mongoose เป็น validator ที่ตอบ 422 เองหรือไม่?

ช่วงที่ 4 / 4

พิสูจน์ด้วยหลักฐาน

อย่าจบที่โค้ด: ปิด เปิด และตามหนังสือเล่มเดียวกันให้ครบเส้นทาง

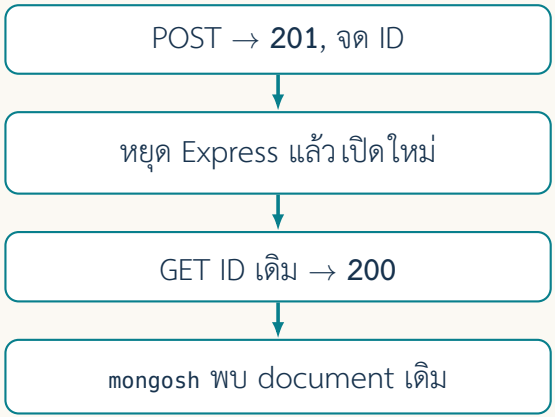
React → Express → MongoDB → Express → React

การทดลองสำคัญ: ปิด Express แล้วหนังสือยังอยู่

ส่ง POST แล้วจด `id` จาก `Location`. GET ด้วย ID นี้ให้เห็นชื่อหนังสือ

หยุด **เฉพาะ Express** แล้วเปิดใหม่. GET ด้วย ID เดิมอีกครั้ง จากนั้นเปิด `mongosh` และหาหนังสือเล่มเดียวกัน

อย่าใช้ `docker compose down -v` เพราะ `-v` ลบ volume ที่เก็บข้อมูล



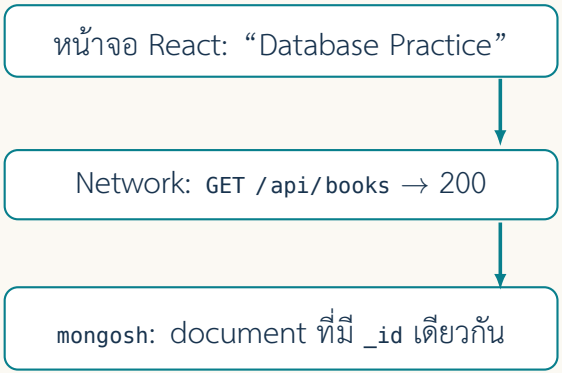
ลองคิดหรือทดลอง: หลักฐานใดพิสูจน์ว่าเล่มนั้นมาจาก MongoDB ไม่ใช่ array ใหม่หลัง restart?

กลับไปหน้า React แล้วดู Network

เปิด React/Vite เดิมและ Express คนละ terminal. หน้า React เรียก `GET /api/books` ผ่าน API service

เปิด DevTools → Network ตรวจ URL, status และ JSON. ดูหนังสือที่เพิ่งเพิ่มบนหน้า แล้วตรวจ `id` เดียวกันใน MongoDB

React example เดิมใช้ `id: string`. เมื่อ API ส่ง ObjectId เป็นข้อความแล้ว ไม่ต้องแปลงจาก number อีก



ลองคิดหรือทดลอง: ถ้าหน้า React ว่าง แต่ `mongosh` มีหนังสือ จะตรวจจุดใดก่อน?

ส่งหลักฐานและอธิบายได้ด้วยตนเอง

ส่ง Book model, store, seed script, `compose.yaml`, `.env.example` และ data dictionary. ห้ามส่ง `.env` หรือข้อมูลรับรองจริง

แนบผล curl: GET list/detail, POST สำเร็จ, ชื่อซ้ำ 409, ข้อมูลผิด 422, ID ไม่พบ 404 และกรณี MongoDB หยุด

หลักฐานสำคัญที่สุดคือ POST → restart Express → GET พบ ID เดิม และ document เดียวกันใน `mongosh`

โค้ด
model + store + routes + seed

ผล จริง
status + JSON + request ID

คำ อธิบาย
ข้อมูล อยู่ ที่ไหน และ ทำไม ยัง อยู่

ลองคิดหรือทดลอง: ถ้าผู้สอนถามว่า “หนังสืออยู่ที่ไหนหลังปิด Express?” คุณจะชี้หลักฐานใด?

อ่านต่อเมื่อทดลองแกนหลักสำเร็จ

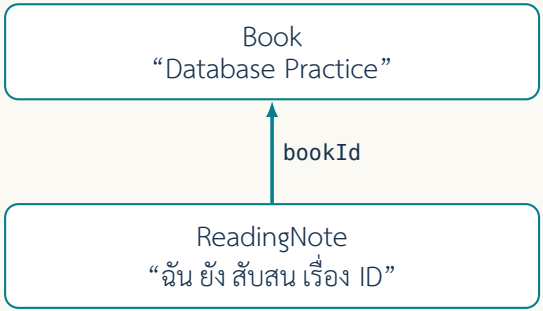
จาก Book ไป ReadingNote

บันทึกการอ่านอาจมี `bookId` อ้างถึงหนังสือหนึ่งเล่ม. ลองคิดก่อนว่าจะฝัง note ใน Book หรือแยก collection เพราะการใช้งานของสองแบบต่างกัน

เมื่อมีหลาย document ที่ต้องเปลี่ยนพร้อมกัน จึงค่อยเรียนเรื่อง transaction จากโจทย์จริง

อ่านประกอบ

หนังสือบทที่ 9: data model, index, transaction
Mongoose schema guide
Mongoose validation
MongoDB unique indexes



เริ่มจากการใช้งานจริง แล้วจึงเลือกแบบข้อมูล

คำถามทบทวน: Mongoose schema กับ MongoDB unique index ช่วยกันตรวจเงื่อนไขของข้อมูลอย่างไร?